

The background is a dark gray surface covered with a complex network of thin, light gray lines. Scattered throughout this network are numerous circular nodes of various sizes and colors, including blue, red, orange, purple, and black. Some nodes are larger than others, and the lines connect them in a web-like pattern, creating a sense of interconnectedness and complexity.

Okres 2-3 Korekta aplikacji i przesyłanie konfiguracji

Znalezione uchybienia

Program posiadał dwa różne timery w głównym oknie, przez to aplikacja próbowała na siłę "doganiać" uciekający czas. Zamiast liczyć płynnie, wykonywała po kilka kroków obliczeniowych naraz, co całkowicie psuło matematykę regulatora PID oraz płynność wykresów.

```
void MainWindow::krokSymulacji()
{
    double dt_gui = intervalMs / 1000.0;

    double czasRzeczywistyTarget = czasBazy + (licznikCzasuRzeczywistego.elapsed() / 1000.0);

    double ost_w = 0.0;
    double ost_y = 0.0;
    double ost_e = 0.0;

    int maxKrokow = 20;
    int wykonaneKroki = 0;

    while (aktualnyCzas < czasRzeczywistyTarget && wykonaneKroki < maxKrokow) {
        aktualnyCzas += dt_gui;

        ost_w = gen.generuj(aktualnyCzas);
        ost_y = petla.wykonaj_krok(ost_w, dt_gui);
        ost_e = ost_w - ost_y;

        wykonaneKroki++;
    }

    if (aktualnyCzas < czasRzeczywistyTarget - dt_gui) {
        aktualnyCzas = czasRzeczywistyTarget;
    }

    if (wykonaneKroki > 0) {
        double u = pid.pobierzOstatnieP() + pid.pobierzOstatnieI() + pid.pobierzOstatnieD();
        aktualizujDaneWykresow(aktualnyCzas, ost_w, ost_y, ost_e, u);
    }
}
```

```
private:
    ModelARX arx;
    RegulatorPID pid;
    Generator gen;
    Petla_Sprzezenia petla;

    QTimer *timerSymulacji;

    QElapsedTimer licznikCzasuRzeczywistego;
    double czasBazy;
    double aktualnyCzas;

    bool czyDziala;
    int intervalMs;
    double oknoCzasowe;
```

```
private:
    ModelARX &model;
    RegulatorPID &regulator;
    Generator &generator;

    QTimer *m_timer;
    int m_intervalMs = 100;
    double m_aktualnyCzas = 0.0;

    void ProstyUAR::onTimeout()
    {
        double dt = m_intervalMs / 1000.0;
        m_aktualnyCzas += dt;

        double w = generator.generuj(m_aktualnyCzas);
        double y = symuluj(w, dt);
        double e = w - y;

        double u = regulator.pobierzOstatnieP()
            + regulator.pobierzOstatnieI()
            + regulator.pobierzOstatnieD();

        emit krokWykonany(m_aktualnyCzas, w, y, e, u);
    }
}
```

Znalezione uchybienia

Program posiadał dwa niemal identyczne pliki realizujące te same obliczenia (jeden historycznie służył do ręcznych testów, drugi obsługiwał czas). Zmiana czegokolwiek wymagała podwójnej pracy i groziła błędami. Połączyliśmy je w jedną solidną klasę, oraz do tej klasy został przeniesiony timer aby odzielić interfejs od obliczeń.

```
#ifndef PROSTYUAR_H
#define PROSTYUAR_H

#include "ModelARX.h"

class RegulatorPID;

class ProstyUAR
{
public:
    ProstyUAR(ModelARX &arx, RegulatorPID &pid);

    // Główna funkcja symulacji pętli zamkniętej
    // w - wartość zadana
    // Zwraca y - wyjście z obiektu
    double symuluj(double w);

private:
    ModelARX &model;
    RegulatorPID &regulator;
    double poprzednie_y; // Do obliczenia uchybu  $e = w - y(k-1)$ 
};

#endif // PROSTYUAR_H

#include "Petla_Sprzezenia.h"

Petla_Sprzezenia::Petla_Sprzezenia(ModelARX &arx, RegulatorPID &pid)
    : model(arx)
    , regulator(pid)
    , ostatnie_wyjscie(0.0)
{}

double Petla_Sprzezenia::wykonaj_krok(double wartosc_zadana, double dt)
{
    double e = wartosc_zadana - ostatnie_wyjscie;
    double u = regulator.symuluj(e, dt);
    double y = model.symuluj(u);
    ostatnie_wyjscie = y;
    return y;
}
```

```
1  #include "ProstyUAR.h"
2
3  ProstyUAR::ProstyUAR(ModelARX &arx, RegulatorPID &pid, Generator &gen, QObject *parent)
4      : QObject(parent)
5        , model(arx)
6        , regulator(pid)
7        , generator(gen)
8        , m_timer(new QTimer(this))
9        , m_interwalMs(100)
10       , m_aktualnyCzas(0.0)
11       , m_ostatnieWyjscie(0.0)
12   {
13       m_timer->setTimerType(Qt::PreciseTimer);
14       connect(m_timer, &QTimer::timeout, this, &ProstyUAR::onTimeout);
15   }
16
17   void ProstyUAR::onTimeout()
18   {
19       double dt = m_interwalMs / 1000.0;
20       m_aktualnyCzas += dt;
21
22       double w = generator.generuj(m_aktualnyCzas);
23       double y = symuluj(w, dt);
24       double e = w - y;
25
26       double u = regulator.pobierzOstatnieP()
27                 + regulator.pobierzOstatnieI()
28                 + regulator.pobierzOstatnieD();
29
30       emit krokWykonany(m_aktualnyCzas, w, y, e, u);
31   }
32
33   double ProstyUAR::symuluj(double wartosc_zadana, double dt)
```

Nawiązywanie połączenia

Funkcjonalność związana z nawiązywaniem połączenia zaczyna się w momencie kliknięcia przycisków wyboru trybu Tryb regulatora lub Tryb obiektu

```
void MainWindow::włączTrybRegulator()
{
    sieci.ustawParametry(static_cast<quint16>(spinPort->value()), edycjaIp->text());
    sieci.set_tryb(Sieci::Tryb::Regulator);
    aktualizujStanKontrolerek();
}

void MainWindow::włączTrybObiekt()
{
    sieci.ustawParametry(static_cast<quint16>(spinPort->value()), edycjaIp->text());
    sieci.set_tryb(Sieci::Tryb::Obiekt);
    aktualizujStanKontrolerek();
}
```

Funkcja do ręcznego zrywania połączenia

```
void MainWindow::rozłączSieć()
{
    sieci.rozłącz();
    aktualizujStanKontrolerek();
}
```

Automatyczna reakcja na zerwanie połączenia

```
void MainWindow::onSieciPolaczono()
{
    ustawWskaźnikPolaczenia(true);
    aktualizujStanKontrolerek();

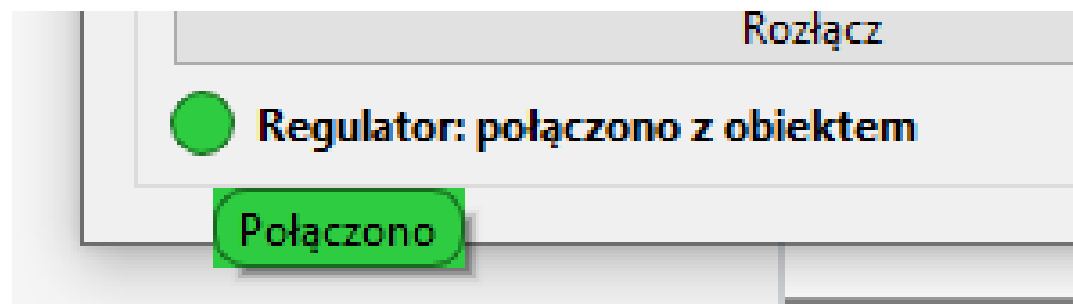
    if (sieci.get_tryb() == Sieci::Tryb::Regulator) {
        sieci.wyslijKonfiguracje(zbudujKonfiguracjeJson());
    }
}

void MainWindow::onSieciRozlaczone()
{
    ustawWskaźnikPolaczenia(false);
    aktualizujStanKontrolerek();
}

void MainWindow::onSieciStatusZmieniony(const QString &opis)
{
    lblStatusSieci->setText(opis);
}

void MainWindow::onSieciOdebranoKonfiguracje(const QJsonObject &konfig)
{
    zastosujKonfiguracjeJson(konfig);
}
```

Sygnalizacja połączenia za pomocą Gui



```
QHBoxLayout *layoutStatus = new QHBoxLayout();
lblWskaznikPolaczenia = new QLabel();
lblWskaznikPolaczenia->setFixedSize(18, 18);
lblWskaznikPolaczenia->setStyleSheet(
    "background-color: #b0b0b0; border-radius: 9px; border: 1px solid #000000ff;");
```

```
void MainWindow::ustawWskaznikPolaczenia(bool polaczony)
{
    if (polaczony) {
        lblWskaznikPolaczenia->setStyleSheet(
            "background-color: #2ecc40; border-radius: 9px; border: 1px solid #186a1f;");
        lblWskaznikPolaczenia->setToolTip("Połączono");
    } else {
        lblWskaznikPolaczenia->setStyleSheet(
            "background-color: #b0b0b0; border-radius: 9px; border: 1px solid #555;");
        lblWskaznikPolaczenia->setToolTip("Rozłączono");
    }
}
```

Blokowanie kontrolek

```
551
552 void MainWindow::aktualizujStanKontrolkek()
553 {
554     Sieci::Tryb tryb = sieci.get_tryb();
555     bool polaczony = sieci.czyPolaczony();
556     bool trybWybrany = (tryb != Sieci::Tryb::Nieokreslony);
557     bool regulator = (tryb == Sieci::Tryb::Regulator);
558     bool obiekt = (tryb == Sieci::Tryb::Obiekt);
559     bool mozePID = !trybWybrany || regulator;
560     bool mozeARX = !trybWybrany || obiekt;
561
562     btnTrybRegulator->setEnabled(!trybWybrany);
563     btnTrybObiekt->setEnabled(!trybWybrany);
564     btnRozlacz->setEnabled(trybWybrany);
565     edycjaIp->setEnabled(!trybWybrany);
566     spinPort->setEnabled(!trybWybrany);
567
568     grpPid->setEnabled(mozePID);
569     grpGen->setEnabled(mozePID);
570     btnArx->setEnabled(mozeARX);
571     btnStartStop->setEnabled(mozePID && (!trybWybrany || polaczony));
572     btnReset->setEnabled(mozePID);
573     spinInterwal->setEnabled(mozePID);
574     btnWczytajJson->setEnabled(mozePID);
575 }
576
```

Symulacja

Zakres Osi X: 10,00 s

Interwał: 200 ms

Start

Pełny Reset

Konfiguracja Modelu ARX...

Regulator PID

Kp: 0.5

Ti: 5.0

Td: 0.2

Metoda I: Stała przed sumą

Reset Pamięci Całki

Wartość Zadana

Kształt: Prostokąt

Składowa stała: 0,00

Amplituda: 1,00

Okres (s): 10,00

Wypełnienie (0-1): 0,50

Czas aktywacji (s): 1,00

Zapisz Konfigurację (JSON)

Wczytaj Konfigurację (JSON)

Sieci

Adres IP: 127.0.0.1

Port: 45763

Tryb regulatora

Tryb

Rozłącz

Błąd sieci: Connection refused

Przekazywanie konfiguracji programu

```
QJsonObject MainWindow::zbudujKonfiguracjeJson() const
{
    QJsonObject root;

    QJsonObject jArx;
    QJsonArray arrA, arrB;
    for (double v : arx.getA())
        arrA.append(v);
    for (double v : arx.getB())
        arrB.append(v);
    jArx["A"] = arrA;
    jArx["B"] = arrB;
    jArx["k"] = arx.getOpoznienie();
    jArx["szum"] = arx.getSzum();
    root["ARX"] = jArx;

    QJsonObject jPid;
    jPid["Kp"] = pid.getKp();
    jPid["Ti"] = pid.getTi();
    jPid["Td"] = pid.getTd();
    jPid["Metoda"] = (int) pid.getMetodaCalkowania();
    root["PID"] = jPid;

    QJsonObject jGen;
    jGen["Typ"] = (int) gen.getTyp();
    jGen["Pocz"] = gen.getPoczek();
    jGen["Zm"] = gen.getZmiana();
    jGen["Czas"] = gen.getCzasZmiany();
    jGen["Wyp"] = gen.getWypelnienie();
    jGen["Akt"] = gen.getCzasAktywacji();
    root["Gen"] = jGen;

    root["Interwal"] = intervalMs;
    root["OknoCzasowe"] = oknoCzasowe;

    return root;
}
```

[TYP PAKIETU]	[ROZMIAR DANYCH]	[WŁAŚCIWE DANE (PAYLOAD)]
(1 bajt)	(4 bajty)	(N bajtów)

```
void Sieci::wyslijPakiet(TypPakietu typ, const QByteArray &payload)
{
    if (!czyPolaczony())
        return;

    QByteArray naglowek;
    QDataStream out(&naglowek, QIODevice::WriteOnly);
    out.setVersion(QDataStream::Qt_5_15);
    out << static_cast<quint8>(typ);
    out << static_cast<quint32>(payload.size());
    m_socket->write(naglowek);
    m_socket->write(payload);
    m_socket->flush();
}
```

```
void Sieci::parsujBufor()
{
    const int ROZMIAR_NAGLOWKA = sizeof(quint8) + sizeof(quint32);

    while (m_bufor.size() >= ROZMIAR_NAGLOWKA) {
        QDataStream in(m_bufor);
        in.setVersion(QDataStream::Qt_5_15);

        quint8 typ;
        quint32 rozmiar;
        in >> typ >> rozmiar;

        if (m_bufor.size() < ROZMIAR_NAGLOWKA + (int) rozmiar) {
            return;
        }

        QByteArray payload = m_bufor.mid(ROZMIAR_NAGLOWKA, rozmiar);
        m_bufor.remove(0, ROZMIAR_NAGLOWKA + rozmiar);

        if (typ == PAKIET_KONFIG) {
            QJsonDocument doc = QJsonDocument::fromJson(payload);
            if (doc.isObject()) {
                qDebug() << "Sieci: odebrano konfigurację";
                emit odebranoKonfiguracje(doc.object());
            }
        }
    }
}
```